

复试机试笔记

- 考研机试练习Github链接: https://github.com/Sailors111/ProgramTest_kaoyan
- C++万能头文件<bits/stdc++.h>

代码块

```
1  #include <bits/stdc++.h>
2  using namespace std;
```

1. 枚举与模拟

1.1 输入输出

- printf函数与scanf函数:

代码块

```
1  int main(){
2      // %d: int、%c: char、%lf: double、%lld: long long、%s: const char*/char[]
3      int value = 0;
4      scanf("value=%d", &value); // 要求输入为value=x, 才能成功赋值
5      printf("value=%d\n", value); // 输出value=x, x为输入的整数
6      double d = 0.222222;
7      printf("%.2lf", d); // 保留小数点后2位
8  }
```

- 输入输出流cin/cout:

代码块

```
1  #include <iostream>
2  #include <string>
3  using namespace std;
4
5  int main(){
6      int n;
7      while(cin >> n){ // 循环输入单个字符串, 不含空格、换行符
8          cout << n << endl;
9      }
10     string str;
```

```

11     while(getline(cin, str)){ // 循环输入一行字符串，包含空格、换行符，需导入
        <string>库
12         cout << str << endl;
13     }
14 }

```

1.2 日期问题

- 判断闰年与获取天数：

代码块

```

1 // 获取某年某月的天数
2 const int days[13] = {0, 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
3
4 // 判断闰年
5 bool isLeapYear(int year){
6     return (year % 4 == 0 && year % 100 != 0) || (year % 400 == 0);
7 }
8
9 // 该月份的总天数
10 int getDaysOfMonth(int year, int month){
11     if(month == 2 && isLeapYear(year)){
12         return 29;
13     }
14     return days[month];
15 }
16
17 // 该年份的总天数
18 int getDaysOfYear(int year) {
19     return isLeapYear(year) ? 366 : 365;
20 }

```

- 日期与总天数的相互转换：（这里的总天数是指从公元1年1月1日到指定年月日的总天数）

代码块

```

1 // 日期转总天数
2 long long dateToDays(int year, int month, int day){ //y-m-d转1-1-1到y-m-d之间的总天数
3     long long res = 0;
4     for(int i = 1; i < year; i++){
5         res += getDaysOfYear(i);
6     }
7     for(int i = 1; i < month; i++){
8         res += getDaysOfMonth(year, i);

```

```

9      }
10     res += day;
11     return res;
12 }
13
14 // 总天数转日期
15 void daysToDate(long long days, int& year, int& month, int& day){ // 1-1-1到
    y-m-d之间的总天数转y-m-d
16     year = 1;
17     while(true){
18         int yd = getDaysOfYear(year);
19         if(days > yd){
20             days -= yd;
21             year++;
22         }else{
23             break;
24         }
25     }
26     month = 1;
27     while(true){
28         int md = getDaysOfMonth(year, month);
29         if(days > md){
30             days -= md;
31             month++;
32         }else{
33             break;
34         }
35     }
36     day = days;
37 }

```

- 日期相差天数：

代码块

```

1 // 两个日期相差天数
2 long long diffDays(int y1, int m1, int d1, int y2, int m2, int d2){
3     long long days1 = dateToDays(y1, m1, d1);
4     long long days2 = dateToDays(y2, m2, d2);
5     return abs(days1 - days2);
6 }

```

- 求星期几：

代码块

```

1 // 求星期几
2 int getWeek(int year, int month, int day) {
3     long long base = dateToDays(2026, 2, 15); // 2026年2月15日为星期日
4     long long now = dateToDays(year, month, day);
5     int wd = ((now - base) % 7 + 7) % 7;
6     return wd;
7 }

```

- 打印日期:

代码块

```

1 // 打印日期
2 void printDate(int year, int month, int day){
3     cout << year << "-";
4     if(month < 10){
5         cout << "0";
6     }
7     cout << month << "-";
8     if(day < 10){
9         cout << "0";
10    }
11    cout << day << endl;
12 }

```

2. 排序与查找

2.1 排序

2.1.1 sort函数

代码块

```

1 bool compare(int x, int y){ // 逆序
2     return x > y;
3 }
4
5 int main(){
6     int len;
7     cin >> len;
8     // 数组排序
9     int* arr = new int[len];
10    for(int i = 0; i < len; i++){
11        arr[i] = len - i;
12    }

```

```

13     sort(arr, arr + len); // 正序排序
14     sort(arr, arr + len, compare); // 逆序排序
15     // 向量排序
16     vector<int> vec;
17     for(int i = 0; i < len; i++){
18         vec.push_back(len - i);
19     }
20     sort(vec.begin(), vec.end()); // 正序排序
21     sort(vec.begin(), vec.end(), compare); // 逆序排序
22 }

```

2.1.2 reverse函数

代码块

```

1  int main(){
2      // 反转数组
3      vector<int> vec;
4      for(int i = 0; i < len; i++){
5          vec.push_back(len - i);
6      }
7      reverse(vec.begin(), vec.end());
8      // 反转字符串
9      string str;
10     cin >> str;
11     reverse(str.begin(), str.end());
12 }

```

2.2 查找

2.2.1 最值函数

代码块

```

1  int main(){
2      vector<int> vec;
3      for(int i = 0; i < len; i++){
4          vec.push_back(len - i);
5      }
6      if(!vec.empty()){
7          int max_val = *max_element(vec.begin(), vec.end()); // 最大值
8          int min_val = *min_element(vec.begin(), vec.end()); // 最小值
9      }
10 }

```

2.2.2 find函数

代码块

```
1  int main(){
2      int len, val;
3      cin >> len >> val;
4      vector<int> vec;
5      for(int i = 0; i < len; i++){
6          vec.push_back(len - i);
7      }
8      if(!vec.empty()){
9          // find(begin, end, val): 在begin和end之间查找val, 找到返回对应迭代器, 找不到返回end
10         auto it = find(vec.begin(), vec.end(), val); // 查找元素, 查找失败返回vec.end()
11         if(it != vec.end()){
12             cout << *it << endl;
13         }
14     }
15 }
```

3. 字符串

3.1 字符串string

- 字符串遍历:

代码块

```
1  int main(){
2      string str, S, T;
3      cin >> str >> S >> T;
4
5      // 遍历
6      for(int i = 0; i < str.size(); i++){ // 普通遍历
7          cout << str[i] << " ";
8      }
9      cout << endl;
10
11     for(string::iterator it = str.begin(); it != str.end(); it++){ // 迭代器遍历
12         cout << *it << " ";
13     }
14     cout << endl;
15
16     for(auto & it : str){ // 智能指针遍历
```

```
17         cout << it << " ";
18     }
19     cout << endl;
20 }
```

- 字符串与基本数据类型的转换：

代码块

```
1  int main(){
2      string str;
3      cin >> str;
4      // 字符串 -> 数字
5      int num_i = stoi(str);
6      long num_l = stol(str);
7      double num_d = stod(str);
8
9      // 数字 -> 字符串
10     string str = to_string(num);
11 }
```

- 相关字段获取：

代码块

```
1  int main(){
2      string str;
3      cin >> str;
4      // 字符串长度
5      cout << str.size() << endl;
6      // 字符串判空
7      cout << str.empty() << endl;
8  }
```

- 拼接与比较：

代码块

```
1  int main(){
2      string str, S, T;
3      cin >> str >> S >> T;
4      // 字符串拼接
5      str += S;
6      cout << str << endl;
7      // 字符串比较
```

```

8      cout << (S < T) << " ";
9      cout << (S > T) << " ";
10     cout << (S == T) << " ";
11     cout << (S != T) << " ";
12     cout << endl;
13 }

```

- 查找子串：

代码块

```

1  int main(){
2      string str, S;
3      cin >> str >> S;
4      // 字符串查找find(str, pos = 0)、rfind(str, pos = len -1)
5      int pos;
6      cin >> pos;
7
8      /* 正向查找 */
9      // find(S): 从str[0]开始查找字符串S, 找到返回第一个字符串的下标, 找不到返回
      string::npos
10     cout << str.find(S) << endl;
11     // find(S, pos): 从str[pos]开始查找字符串S, 找到返回第一个字符串的下标, 找不到
      返回string::npos
12     cout << str.find(S, pos) << endl;
13
14     /* 反向查找 */
15     // rfind(S): 从字符串str的末尾开始反向查找子串S, 找到则返回S最后一次出现的起始下
      标, 找不到返回string::npos
16     cout << str.rfind(S) << endl;
17     // rfind(S, pos): 从str的pos下标位置开始 (向前) 反向查找子串S, 找到则返回S在
      [0,pos]范围内最后一次出现的起始下标, 找不到返回string::npos
18     cout << str.rfind(S, pos) << endl;
19 }

```

- 截取子串：

代码块

```

1  int main(){
2      string str, S;
3      cin >> str >> S;
4      // 取字符串子串substr(pos, len)
5      int pos = str.find(S), len = S.size();
6      if(pos != string::npos){

```

```

7         string substr = str.substr(pos, len); // 从str[pos]开始, 取长度为len的
           子串
8         cout << substr << endl;
9     }
10 }

```

- 替换子串：

代码块

```

1  int main(){
2      string str, S, T;
3      cin >> str >> S >> T;
4      // 字符串替换replace(pos, len, str)
5      int pos = str.find(S), len = S.size();
6      if(pos != string::npos){
7          str.replace(pos, len, T); // 从str[pos]开始, 把长度为len的子串替换为T
8          cout << str << endl;
9      }
10 }

```

- 插入子串：

代码块

```

1  int main(){
2      string str;
3      cin >> str;
4      // 字符串插入insert(index, substr): 在index开始插入substr
5      str.insert(1, "hello");
6      // 字符串插入insert(index, n, ch): 在index开始插入n个字符ch
7      str.insert(1, 3, 'w');
8  }

```

- 删除子串与清空：

代码块

```

1  int main(){
2      string str, S, T;
3      cin >> str >> S >> T;
4      // 删除字符串子串erase(pos, len)
5      int pos = str.find(T), len = T.size();
6      if(pos != string::npos){
7          str.erase(pos, len); // 从str[pos]开始, 删除长度为len的子串

```

```

8         cout << str << endl;
9     }
10    // 字符串清空
11    str.clear();
12    cout << str << endl;
13 }

```

3.2 字符串处理

- 字符串替换：

代码块

```

1 // 字符串替换
2 void replaceAll(string& str, const string& S, const string& T){ // 将str中所
   有的S替换为T
3     if(S.empty()){
4         return;
5     }
6     size_t pos = 0;
7     while((pos = str.find(S, pos)) != string::npos){
8         str.replace(pos, S.size(), T);
9         pos += T.size();
10    }
11 }

```

- 字符串分割：

代码块

```

1 // 字符串分割
2 vector<string> split(string str, char ch) { // 将str按ch为界分割
3     vector<string> vec;
4     int start = 0, len = str.size();
5     for (int end = 0; end < len; end++) {
6         if (str[end] == ch) {
7             vec.push_back(str.substr(start, end - start));
8             while (end < len && str[end] == ch) {
9                 end++;
10            }
11            start = end;
12        }
13    }
14    if (start < len) {
15        vec.push_back(str.substr(start));
16    }

```

```
17     return vec;
18 }
```

- 字符串大小写：

代码块

```
1  // 字符串大小写
2  void toLower(string& str){    // 将字符串中的大写字母转为小写字母
3      for(int i = 0; i < str.size(); i++){
4          if(str[i] >= 'A' && str[i] <= 'Z'){
5              str[i] = str[i] - 'A' + 'a';
6          }
7      }
8  }
9
10 void toUpper(string& str){    // 将字符串中的小写字母转为大写字母
11     for(int i = 0; i < str.size(); i++){
12         if(str[i] >= 'a' && str[i] <= 'z'){
13             str[i] = str[i] - 'a' + 'A';
14         }
15     }
16 }
```

- 浮点数加法：

代码块

```
1  // 浮点数加法
2  void InitProcess(string& S, string& T) {
3      int pos01 = S.find('.'), pos02 = T.find('.');
4      while (pos01 < pos02) {
5          S = '0' + S;
6          pos01++;
7      }
8      while (pos02 < pos01) {
9          T = '0' + T;
10         pos02++;
11     }
12     int len01 = S.size(), len02 = T.size();
13     while (len01 < len02) {
14         S = S + '0';
15         len01++;
16     }
17     while (len02 < len01) {
18         T = T + '0';
19     }
20 }
```

```
19         len02++;
20     }
21 }
22
23 string FloatAdd(string S, string T) {
24     InitProcess(S, T);
25     int carry = 0;
26     int len = S.size();
27     string res = "";
28     for (int i = len - 1; i >= 0; i--) {
29         if (S[i] == '.') {
30             res = '.' + res;
31             continue;
32         } else {
33             int sum = S[i] - '0' + T[i] - '0' + carry;
34             carry = sum / 10;
35             res = char(sum % 10 + '0') + res;
36         }
37     }
38     if (carry > 0) {
39         res = '1' + res;
40     }
41     // 把字符串首部的所有0去掉
42     int end;
43     for (end = 0; end < res.size(); end++) {
44         if (res[end] != '0') {
45             break;
46         }
47     }
48     res.erase(0, end);
49     if (res[0] == '.') {
50         res = '0' + res;
51     }
52     // 把字符串尾部的所有0去掉
53     for (end = res.size() - 1; end >= 0; end--) {
54         if (res[end] != '0') {
55             break;
56         }
57     }
58     res.erase(end + 1, res.size() - end - 1);
59     if (res[res.size() - 1] == '.') {
60         res = res + '0';
61     }
62     return res;
63 }
```

4. 初级数据结构

4.1 向量vector

- 常规操作：

代码块

```
1  int main(){
2      vector<int> vec;
3      int n, val;
4      cin >> n >> val;
5      // 初始化
6      vec = {0, 1, 2, 3}; // 直接赋值
7      vec = vector<int>(); // 初始化为空向量
8      vec = vector<int>(n); // 长度为n, 全部初始化为0
9      vec = vector<int>(n, val); // 长度为n, 全部初始化为val
10
11     // 普通遍历
12     for(int i = 0; i < vec.size(); i++){
13         cout << vec[i] << " ";
14     }
15     cout << endl;
16     // 迭代器遍历
17     for(vector<int>::iterator it = vec.begin(); it != vec.end(); it++){
18         cout << *it << " ";
19     }
20     cout << endl;
21     // 智能指针遍历
22     for(auto & it : vec){
23         cout << it << " ";
24     }
25     cout << endl;
26
27     // 插入
28     for(int i = 0; i < n; i++){
29         vec.push_back(i); // 在末尾插入元素
30     }
31     // insert(iterator, val): 在vector<Type>::iterator后插入元素val
32     vec.insert(vec.end(), n); // 在vec.end()后插入元素
33
34     // 删除
35     vec.pop_back(); // 删除末尾元素
36     // erase(iterator): 删除iterator所指的元素
37     vec.erase(vec.begin()); // 删除vec.begin()的元素
38     // erase(iterator, iterator+n): 删除[iterator, iterator + n)所指的元素
```

```

39     vec.erase(vec.begin(), vec.begin() + n); // 删除[vec.begin(),
        vec.begin() + n)区间内的元素
40     // 清空
41     vec.clear();
42 }

```

- 数组的应用-二分查找：

代码块

```

1  // 二分查找
2  int binary_search(const vector<int>& items, int target){
3      int left = 0, right = items.size() - 1;
4      while(left <= right){
5          int mid = left + (right - left) / 2;
6          if(items[mid] == target){
7              return mid;
8          }else if(items[mid] < target){
9              left = mid + 1;
10         }else{
11             right = mid - 1;
12         }
13     }
14     return -1;
15 }

```

- 数组的应用-两数之和：

代码块

```

1  // 两数之和
2  vector<vector<int>> twoSum(vector<int> items, int target) {
3      vector<vector<int>> res;
4      sort(items.begin(), items.end());
5      int left = 0, right = items.size() - 1;
6      while (left <= right) {
7          if (items[left] + items[right] == target) {
8              res.push_back({items[left], items[right]});
9              left++;
10             right--;
11         } else if (items[left] + items[right] < target) {
12             left++;
13         } else {
14             right--;
15         }
16     }

```

```
17     return res;
18 }
```

- 数组的应用-三数之和：

代码块

```
1  // 三数之和
2  vector<vector<int>> threeSum(vector<int> items, int target){
3      vector<vector<int>> res;
4      sort(items.begin(), items.end());
5      int len = items.size();
6      for(int i = 0; i < len; i++){
7          if(i > 0 && items[i] == items[i-1]){
8              continue;
9          }
10         int left = i + 1, right = len - 1;
11         while(left < right){
12             if(items[i] + items[left] + items[right] == target){
13                 res.push_back({items[i], items[left], items[right]});
14                 while(left < right && items[left] == items[left+1]){
15                     left++;
16                 }
17                 while(left < right && items[right] == items[right-1]){
18                     right--;
19                 }
20                 left++;
21                 right--;
22             }else if(items[i] + items[left] + items[right] < target){
23                 left++;
24             }else{
25                 right--;
26             }
27         }
28     }
29     return res;
30 }
```

4.2 队列queue

- 常规操作：

代码块

```
1  int main(){
2      queue<int> q;
```

```

3      int n;
4      cin >> n;
5      // 入队
6      for(int i = 0; i < n; i++){
7          q.push(i);
8      }
9      // 出队
10     q.pop();
11     // 取队头
12     int front = q.front();
13     // 取队尾
14     int back = q.back();
15     // 判队空
16     cout << q.empty() << endl;
17 }

```

4.3 栈stack

- 常规操作：

代码块

```

1  int main(){
2      stack<int> st;
3      int n;
4      cin >> n;
5      // 入栈
6      for(int i = 0; i < n; i++){
7          st.push(i);
8      }
9      // 出栈
10     st.pop();
11     // 取栈顶
12     int top = st.top();
13     // 判栈空
14     cout << st.empty() << endl;
15 }

```

- 栈的应用-括号匹配：

代码块

```

1  // 栈的应用-括号匹配
2  bool isMatch(string str){
3      stack<char> st;
4      for(int i = 0; i < str.size(); i++){

```

```

5         // 左括号入栈
6         if(str[i] == '(' || str[i] == '[' || str[i] == '{'){
7             st.push(str[i]);
8             continue;
9         }
10        // 右括号判断
11        if(str[i] == ')'){
12            if(st.empty() || st.top() != '('){
13                return false;
14            }
15            st.pop();
16        }
17        if(str[i] == ']'){
18            if(st.empty() || st.top() != '['){
19                return false;
20            }
21            st.pop();
22        }
23        if(str[i] == '}'){
24            if(st.empty() || st.top() != '{'){
25                return false;
26            }
27            st.pop();
28        }
29    }
30    return st.empty();
31 }
32

```

- 栈的应用-表达式求值：

代码块

```

1  // 栈的应用-表达式求值
2  int priority(char ch){        // 判断操作符的优先级
3      if(ch == '#'){
4          return 0;
5      }else if(ch == '$'){
6          return 1;
7      }else if(ch == '+' || ch == '-'){
8          return 2;
9      }else{
10         return 4;
11     }
12 }
13

```

```

14 double calculate(double x, double y, char op){    // 操作数计算
15     double res;
16     if (op == '+') {
17         res = x + y;
18     } else if (op == '-') {
19         res = x - y;
20     } else if (op == '*') {
21         res = x * y;
22     } else if (op == '/') {
23         res = x / y;
24     }
25     return res;
26 }
27
28 double getNumber(string str, int& index){    // 从字符串中读数字
29     double number = 0;
30     while(isdigit(str[index])){    // isdigit函数用于判断字符是不是数字
31         number = number * 10 + str[index] - '0';
32         index++;
33     }
34     return number;
35 }
36
37 double calculateExpression(string str){    // 表达式求值
38     int index = 0;
39     stack<char> opStack;
40     stack<double> numStack;
41     str += "$";
42     opStack.push('#');
43     while(index < str.size()){
44         if(str[index] == ' '){
45             index++;
46         } else if(isdigit(str[index])){
47             numStack.push(getNumber(str, index));
48         } else{
49             if(priority(opStack.top()) < priority(str[index])){ // 栈顶操作符
// 优先级小于当前操作符
50                 opStack.push(str[index]);
51                 index++;
52             } else{ // 栈顶操作符优先级大于等于当前操作符
53                 double y = numStack.top();
54                 numStack.pop();
55                 double x = numStack.top();
56                 numStack.pop();
57                 numStack.push(calculate(x, y, opStack.top()));
58                 opStack.pop();
59             }

```

```
60     }
61 }
62     return numStack.top();
63 }
```

5. 数学问题

5.1 进制转换

- 预备知识：
 - N进制数加法，N为1~10之间的整数：

代码块

```
1  // N进制数加法，N为1~10的整数
2  string add_N(string x, string y, int N) {
3      int len1 = x.size(), len2 = y.size();
4      // 补0对齐
5      while (len1 < len2) {
6          x = "0" + x;
7          len1++;
8      }
9      while (len2 < len1) {
10         y = "0" + y;
11         len2++;
12     }
13     // 进位加法
14     string res = "";
15     int len = x.size(), carry = 0;
16     for (int i = len - 1; i >= 0; i--) {
17         int sum = x[i] - '0' + y[i] - '0' + carry;
18         carry = sum / N;
19         res = char(sum % N + '0') + res;
20     }
21     if (carry > 0) {
22         res = "1" + res;
23     }
24     return res;
25 }
```

- 十进制数乘以N运算，N为1~10之间的整数：

代码块

```
1  //十进制数乘以N，N是1~10之间的整数
```

```

2  string multiple(string num, int N) {
3      string res = "";
4      int carry = 0;
5      for (int i = num.size() - 1; i >= 0; i--) {
6          int product = (num[i] - '0') * N + carry;
7          carry = product / 10;
8          res = char(product % 10 + '0') + res;
9      }
10     if (carry > 0) {
11         res = char(carry + '0') + res;
12     }
13     return res;
14 }

```

- 十进制数除以N运算，N为1~9之间的整数：

代码块

```

1  // 十进制数除以N, N为1~9之间的整数
2  string divide(string num, int N){
3      string res = "";
4      int carry = 0;
5      // 模拟除法
6      for(int i = 0; i < num.size(); i++){
7          int sum = num[i] - '0' + carry * 10;
8          carry = sum % N;
9          res += char(sum / N + '0');
10     }
11     // 去除前导0
12     int end = 0;
13     while(end < res.size() && res[end] == '0'){
14         end++;
15     }
16     if(end == res.size()){
17         return "0";
18     }
19     res = res.substr(end);
20     return res;
21 }

```

- 十进制数取模N运算，N为1~9之间的整数：

代码块

```

1  // 十进制数取模N, N为1~9之间的整数
2  char mod(string num, int N) {

```

```

3     int res = 0;
4     for (int i = 0; i < num.size(); i++) {
5         res = (res * 10 + (num[i] - '0')) % N;
6     }
7     return char(res + '0');
8 }

```

- 十进制与N进制的转换：

- 十进制转N进制，N为1~9之间的整数：

代码块

```

1 // 十进制转N进制，N为1~9的整数
2 string TenToN(string num, int N) { // num等于1或0时需另外判断
3     string res = "";
4     while (num != "0") {
5         res = mod(num, N) + res;
6         num = divide(num, N);
7     }
8     return res;
9 }

```

- N进制转十进制，N为1~9之间的整数：

代码块

```

1 // N进制转十进制，N为1~9的整数
2 string NToTen(string num, int N) { // num等于1或0时需另外判断
3     string res = "0";
4     for (int i = 0; i < num.size(); i++) {
5         res = multiple(res, N);
6         res = add_N(res, to_string(num[i] - '0'), 10);
7     }
8     return res;
9 }

```

- M进制转N进制：

- 字符与数字转化：

代码块

```

1 // 字符转数字
2 int CharToInt(char ch){
3     int res = 0;

```

```

4     if('0' <= ch && ch <= '9'){
5         res = ch - '0';
6     }else if('a' <= ch && ch <= 'z'){
7         res = ch - 'a' + 10;
8     }else if('A' <= ch && ch <= 'Z'){
9         res = ch - 'A' + 10;
10    }
11    return res;
12 }
13
14 // 数字转字符
15 char IntToChar(int num){
16     char res = ' ';
17     if(num < 10){
18         res = num + '0';
19     }else{
20         res = num - 10 + 'a';
21     }
22     return res;
23 }

```

- M进制转N进制：

代码块

```

1 // M进制转N进制
2 string MToN(string num, int M, int N){
3     string res = "";
4     unsigned long long temp = 0;
5     for(int i = 0; i < num.size(); i++){
6         temp = CharToInt(num[i]) + temp * M;
7     }
8     while(temp != 0){
9         res = IntToChar(temp % N) + res;
10        temp /= N;
11    }
12    if(res.empty()){
13        res = "0";
14    }
15    return res;
16 }

```

5.2 最大公约数与最小公倍数

- 最大公约数：

代码块/ 最大公约数

```
2  int gcd(int a, int b){
3      if(b == 0){                // 辗转相除法
4          return a;
5      }else{
6          return gcd(b, a % b);
7      }
8  }
```

- 最小公倍数：

代码块

```
1  // 最小公倍数
2  int lcm(int a, int b){
3      return a / gcd(a, b) * b;
4  }
```

5.3 质数与分解质因数

- 判断质数：

代码块

```
1  // 判断是否为质数
2  bool isPrime(long long num) {
3      if(num == 2) {          // 2为质数
4          return true;
5      } else if(num < 2 || num % 2 == 0) {    // 小于2的数和不是2的偶数都不是质数
6          return false;
7      } else {
8          for(long long i = 3; i * i <= num; i += 2){    // 从3开始，只判断奇数
9              if(num % i == 0){
10                 return false;
11             }
12         }
13         return true;
14     }
15 }
```

- 分解约数：

代码块

```
1  // 求约数的个数
```

```

2  int numOfDivisor(long long num){
3      int count = 0;
4      long long i;
5      for(i = 1; i * i < num; i++){
6          if(num % i == 0){
7              count += 2;
8          }
9      }
10     if(i * i == num){
11         count++;
12     }
13     return count;
14 }
15
16 // 求所有约数
17 vector<long long> getDivisor(long long num){
18     vector<long long> res;
19     long long i;
20     for(i = 1; i * i < num; i++){
21         if(num % i == 0){
22             res.push_back(i);
23             res.push_back(num / i);
24         }
25     }
26     if(i * i == num){
27         res.push_back(i);
28     }
29     sort(res.begin(), res.end());
30     return res;
31 }

```

- 分解质因数：

代码块

```

1  // 求质因数的个数
2  int numOfPrimeFactor(long long num){
3      int count = 0;
4      for(long long i = 2; i * i <= num; i++){
5          if(num % i == 0){
6              while(num % i == 0){
7                  num /= i;
8                  count++;
9              }
10         }
11     }

```

```

12     if(num > 1){ // num为质数
13         count++;
14     }
15     return count;
16 }
17
18 // 求所有质因数及其指数
19 vector<pair<long long, int>> getPrimeFactor(long long num){
20     vector<pair<long long, int>> res;
21     for(long long i = 2; i * i <= num; i++){
22         if(num % i == 0){
23             int index = 0; // 质因数的指数
24             while(num % i == 0){
25                 num /= i;
26                 index++;
27             }
28             res.emplace_back(i, index);
29         }
30     }
31     if(num > 1){ // num为质数
32         res.emplace_back(num, 1);
33     }
34     return res;
35 }

```

5.4 快速幂

- 快速幂求 $\text{pow}(x, y)$:

代码块

```

1 // 快速幂求 pow(x, y)
2 long long quickPow(long long x, long long y){ // 求pow(x, y)
3     long long res = 1;
4     while(y != 0){
5         if(y % 2 == 1){ // y为奇数
6             res = res * x;
7         }
8         x = x * x;
9         y /= 2;
10    }
11    return res;
12 }

```

- 快速幂求 $\text{pow}(x, y) \% \text{mod}$:

代码块/ 快速幂求 $\text{pow}(x, y) \% \text{mod}$

```
2   int quickPow(long long x, long long y, int mod){    // 求 $\text{pow}(x, y) \% \text{mod}$ 
3       int res = 1;
4       while(y != 0){
5           if(y % 2 == 1){    // y为奇数
6               res = res * x % mod;
7           }
8           x = x * x % mod;
9           y /= 2;
10      }
11      return res;
12  }
```

5.5 矩阵与矩阵快速幂

- 矩阵的定义：

代码块

```
1   // 矩阵的定义
2   struct Matrix {
3       int row;
4       int col;
5       vector<vector<long long>> matrix;
6       Matrix() : row(0), col(0) {
7           matrix = vector<vector<long long>>(0, vector<long long>(0, 0));
8       }
9       Matrix(int r, int c) : row(r), col(c) {
10          matrix = vector<vector<long long>>(r, vector<long long>(c, 0));
11      }
12
13      friend Matrix operator+(const Matrix& A, const Matrix& B) {
14          if (A.row != B.row || A.col != B.col) {
15              return Matrix();
16          }
17          int row = A.row, col = A.col;
18          Matrix C(row, col);
19          for (int i = 0; i < row; i++) {
20              for (int j = 0; j < col; j++) {
21                  C.matrix[i][j] = A.matrix[i][j] + B.matrix[i][j];
22              }
23          }
24          return C;
25      }
26
27      friend Matrix operator-(const Matrix& A, const Matrix& B) {
28          if (A.row != B.row || A.col != B.col) {
```

```

29         return Matrix();
30     }
31     int row = A.row, col = A.col;
32     Matrix C(row, col);
33     for (int i = 0; i < row; i++) {
34         for (int j = 0; j < col; j++) {
35             C.matrix[i][j] = A.matrix[i][j] - B.matrix[i][j];
36         }
37     }
38     return C;
39 }
40
41 friend Matrix operator*(const Matrix& A, const Matrix& B) {
42     if (A.col != B.row) {
43         return Matrix();
44     }
45     int row = A.row, col = B.col, len = A.col;
46     Matrix C(row, col);
47     for (int i = 0; i < row; i++) {
48         for (int j = 0; j < col; j++) {
49             for (int k = 0; k < len; k++) {
50                 C.matrix[i][j] += A.matrix[i][k] * B.matrix[k][j];
51             }
52         }
53     }
54     return C;
55 }
56
57 Matrix operator=(const Matrix& A){
58     this->row = A.row;
59     this->col = A.col;
60     int row = A.row, col = A.col;
61     this->matrix = vector<vector<long long>>(row, vector<long long>
(col, 0));
62     for(int i = 0; i < row; i++){
63         for(int j = 0; j < col; j++){
64             this->matrix[i][j] = A.matrix[i][j];
65         }
66     }
67     return *this;
68 }
69
70 friend istream& operator>>(istream& in, Matrix& A){
71     for(int i = 0; i < A.row; i++){
72         for(int j = 0; j < A.col; j++){
73             in >> A.matrix[i][j];
74         }

```

```

75     }
76     return in;
77 }
78
79 friend ostream& operator<<(ostream& out, const Matrix& A){
80     for(int i = 0; i < A.row; i++){
81         for(int j = 0; j < A.col; j++){
82             out << A.matrix[i][j] << " ";
83         }
84         out << endl;
85     }
86     return out;
87 }
88 };

```

- 求单位矩阵：

代码块

```

1  // 求单位矩阵
2  Matrix unitMatrix(int n){
3      Matrix E(n, n);
4      for(int i = 0; i < n; i++){
5          for(int j = 0; j < n; j++){
6              if(i == j){
7                  E.matrix[i][j] = 1;
8              }
9          }
10     }
11     return E;
12 }

```

- 矩阵快速幂：

代码块

```

1  // 矩阵快速幂
2  Matrix matrixQuickPow(Matrix A, int k){
3      if(A.row != A.col){
4          return Matrix();
5      }
6      int len = A.row;
7      Matrix res = unitMatrix(len);
8      while(k != 0){
9          if(k % 2 == 1){
10             res = res * A;

```

```

11         }
12         A = A * A;
13         k /= 2;
14     }
15     return res;
16 }

```

- 矩阵快速幂取模：

代码块

```

1  // 矩阵快速幂取模操作
2  Matrix matrixMultiply_mod(const Matrix& A, const Matrix& B, int mod) { //
    取模矩阵乘法
3      if (A.col != B.row) {
4          return Matrix(0, 0);
5      }
6      int row = A.row, col = B.col;
7      Matrix C(row, col);
8      for (int i = 0; i < row; i++) {
9          for (int j = 0; j < col; j++) {
10             for (int k = 0; k < A.col; k++) {
11                 C.matrix[i][j] = (C.matrix[i][j] % mod + A.matrix[i][k] %
mod * B.matrix[k][j] % mod) % mod;
12             }
13         }
14     }
15     return C;
16 }
17
18 Matrix matrixQuickPow_mod(Matrix A, int n, int mod) { // 取模矩阵快速幂
19     if (A.row != A.col) {
20         return Matrix(0, 0);
21     }
22     int len = A.row;
23     Matrix result = unitMatrix(len);
24     while (n != 0) {
25         if (n % 2 == 1) {
26             result = matrixMultiply_mod(result, A, mod);
27         }
28         n /= 2;
29         A = matrixMultiply_mod(A, A, mod);
30     }
31     return result;
32 }

```

5.6 高精度整数

- 高精度整数定义：

代码块

```
1  // 高精度无符号整数
2  struct BigInteger{
3      const static int MAX = 10000;
4      int length;
5      int digit[MAX];
6
7      BigInteger(){
8          memset(digit, 0, sizeof(digit));
9          length = 0;
10     }
11
12     BigInteger(long long num){
13         memset(digit, 0, sizeof(digit));
14         int index = 0;
15         if(num == 0){
16             digit[index] = num;
17             index++;
18         }
19         while(num != 0){
20             digit[index] = num % 10;
21             num /= 10;
22             index++;
23         }
24         length = index;
25     }
26
27     BigInteger(string num){
28         memset(digit, 0, sizeof(digit));
29         int index = 0;
30         for(int i = num.size() - 1; i >= 0; i--){
31             digit[index] = num[i] - '0';
32             index++;
33         }
34         length = index;
35     }
36
37     BigInteger(const BigInteger& B){
38         memset(digit, 0, sizeof(digit));
39         length = B.length;
40         for(int i = 0; i < length; i++){
41             digit[i] = B.digit[i];
42         }
43     }
```

```

43     }
44
45     BigInteger operator=(const BigInteger& B){
46         memset(this->digit, 0, sizeof(this->digit));
47         this->length = B.length;
48         for(int i = 0; i < this->length; i++){
49             this->digit[i] = B.digit[i];
50         }
51         return *this;
52     }
53
54     friend BigInteger operator+(const BigInteger& A, const BigInteger& B){
55         int len1 = A.length, len2 = B.length;
56         int len = len1 > len2 ? len1 : len2, index = 0, carry = 0;
57         BigInteger C;
58         while(index < len){
59             int sum = A.digit[index] + B.digit[index] + carry;
60             C.digit[index] = sum % 10;
61             carry = sum / 10;
62             index++;
63         }
64         C.length = index;
65         if(carry > 0){
66             C.digit[C.length++] = 1;
67         }
68         return C;
69     }
70
71     friend istream& operator>>(istream& in, BigInteger& B){
72         string str;
73         in >> str;
74         B = BigInteger(str);
75         return in;
76     }
77
78     friend ostream& operator<<(ostream& out, const BigInteger& B){
79         int len = B.length;
80         for(int i = len - 1; i >= 0; i--){
81             out << B.digit[i];
82         }
83         return out;
84     }
85 };

```

6. 贪心

6.1 区间贪心

- 不重叠区间选择：选择若干个互不重叠的区间，使区间个数最大（选右端点最小的）。

代码块

```
1  bool compare(pair<int, int> x, pair<int, int> y){
2      return x.second < y.second;
3  }
4
5  // 不重叠区间选择（端点不算重叠）
6  int interval_select(const vector<pair<int, int>>& vec){
7      sort(vec.begin(), vec.end(), compare);
8      int count = 1;
9      int last = vec[0].second; // 选最早结束的
10     for(int i = 1; i < vec.size(); i++){
11         if(vec[i].first >= last){
12             count++;
13             last = vec[i].second;
14         }
15     }
16     return count;
17 }
```

- 删除区间：删除若干个区间，使得剩下的区间数量最大且互不重叠（端点相同算重叠）。

代码块

```
1  // 删除区间（端点算重叠）
2  int interval_delete(const vector<pair<int, int>>& vec){
3      sort(vec.begin(), vec.end(), compare);
4      int count = 1;
5      int last = vec[0].second;
6      for(int i = 1; i < vec.size(); i++){
7          if(vec[i].first > last){
8              count++;
9              last = vec[i].second;
10         }
11     }
12     return vec.size() - count;
13 }
```

7. 递归与分治

7.1 递归

- 递归通用模板：递归是函数自己调用自己（一个子问题的分治）。

代码块

```
1  // 递归通用模板
2  返回值 递归函数(参数) {
3      // 1. 递归终止条件
4      if (最小情况/不能再分) {
5          return 结果;
6      }
7
8      // 2. 处理当前层逻辑 (可选)
9      做一点事...
10
11     // 3. 递归调用 (把问题缩小)
12     结果 = 递归函数(缩小后的参数);
13
14     // 4. 合并结果 (可选)
15     整合一下...
16
17     // 5. 返回答案
18     return 结果;
19 }
```

- N的阶乘：

代码块

```
1  // N的阶乘
2  long long Factorial(long long num){
3      if(num == 0){
4          return 1;
5      }else{
6          return num * Factorial(num - 1);
7      }
8  }
```

7.2 分治

- 分治通用模板：分治是把问题拆成两个或多个独立子问题 → 分别解决 → 合并结果。

代码块

```
1  // 分治通用模板
2  返回值 分治函数(参数) {
3      // 1. 终止条件：问题足够小，直接返回结果
```

```

4     if (问题不可再分) {
5         return 基础解;
6     }
7
8     // 2. 分解: 把大问题切成 2 个/多个独立小问题
9     左问题 = 切分左边;
10    右问题 = 切分右边;
11
12    // 3. 解决: 递归处理每个子问题
13    左结果 = 分治函数(左问题);
14    右结果 = 分治函数(右问题);
15
16    // 4. 合并: 把左右结果合成最终答案
17    最终结果 = 合并(左结果, 右结果);
18
19    return 最终结果;
20 }

```

- 斐波那契数列:

代码块

```

1  // Fibonacci数列
2  long long Fibonacci(long long num){
3      if(num == 0 || num == 1){
4          return num;
5      }else{
6          return Fibonacci(num - 1) + Fibonacci(num - 2);
7      }
8  }

```

- 完全二叉树求节点个数:

代码块

```

1  // 完全二叉树求节点数
2  int countNodes(int index, int total_node_num){    // index为节点编号,
                                                    total_node_num为节点总数
3      if(index > total_node_num){
4          return 0;
5      }else{
6          return 1 + countNodes(2 * index, total_node_num) + countNodes(2 *
index + 1, total_node_num);
7      }
8  }

```

8. 搜索

8.1 宽度优先搜索BFS

- BFS通用模板：

代码块

```
1  // BFS 通用模板 (队列, 层序遍历/最短路径)
2  void bfs(起始点) {
3      // 1. 创建队列 + 访问标记
4      queue<节点类型> q;
5      q.push(起始点);
6      visited[起始点] = true;
7
8      // 2. 循环: 队列不为空就继续
9      while (!q.empty()) {
10         // 取出队首
11         节点 cur = q.front();
12         q.pop();
13
14         // 3. 处理当前节点 (输出/记录/判断)
15         处理 cur;
16
17         // 4. 把所有有效邻居加入队列
18         for (遍历所有邻居) {
19             if (邻居 有效 且 未访问) {
20                 visited[邻居] = true;
21                 q.push(邻居);
22             }
23         }
24     }
25 }
```

- 无权图最短路径问题：

代码块

```
1  // 无权图最短路径
2  vector<int> bfs_shortest_path(const vector<vector<int>>& graph, int start){
3      int num = graph.size();
4      vector<int> dist(graph.size(), -1);
5      vector<bool> visited(graph.size(), false);
6      queue<int> q;
7      // 起始点入队
8      q.push(start);
9      dist[start] = 0;
```

```

10     visited[start] = true;
11     while(!q.empty()){
12         // 队头出队
13         int current = q.front();
14         q.pop();
15         // 遍历当前节点的相邻节点
16         for(int next = 0; next < num; next++){
17             if(!visited[next] && graph[current][next] == 1){ // 未访问且存在边
18                 dist[next] = dist[current] + 1;
19                 visited[next] = true;
20                 q.push(next);
21             }
22         }
23     }
24     return dist;
25 }

```

8.2 深度优先搜索DFS

- DFS通用模板：

代码块

```

1  // DFS 通用模板 (递归写法)
2  void dfs(参数) {
3      // 1. 递归终止条件 (不能再往下走了)
4      if (满足终止条件) {
5          记录答案 / 直接返回;
6          return;
7      }
8
9      // 2. 标记当前节点已访问
10     visited[当前] = true;
11
12     // 3. 遍历所有可能的方向/选择 (上下左右 / 子节点 / 下一个数)
13     for (遍历所有邻居/下一个选择) {
14         if (该节点 有效 且 未访问) {
15             dfs(下一个节点); // 递归深入
16         }
17     }
18
19     // 4. 回溯: 撤销标记
20     visited[当前] = false;
21 }

```

- 子集问题：无顺序，选择所有的可能子集（包括空集），收集所有结果。

代码块

```
1 // 子集问题模板
2 void dfs_subset(const vector<int>& items, vector<int>& path, int start,
  vector<vector<int>>& res){
3     res.push_back(path);
4     for(int i = start; i < items.size(); i++){
5         // 选择
6         path.push_back(items[i]);
7         // 递归
8         dfs_subset(items, path, i + 1, res);
9         // 回溯
10        path.pop_back();
11    }
12 }
13
14 vector<vector<int>> subset(const vector<int>& items){
15     vector<int> path;
16     vector<vector<int>> res;
17     dfs_subset(items, path, 0, res);
18     return res;
19 }
```

- 组合问题：无顺序，n个里面选 k 个 ($1 \leq k \leq n$)，不能重复选。

代码块

```
1 // 组合问题模板
2 void dfs_combination(const vector<int>& items, vector<int>& path, int
  start, int target, vector<vector<int>>& res){
3     // items存放待选择的元素、path存放选择的元素、res保存所有满足元素之和为target的
  path
4     if(target == 0){
5         res.push_back(path);
6         return;
7     }else if(target < 0){
8         return;
9     }
10    for(int i = start; i < items.size(); i++){
11        if(items[i] > target){
12            continue;
13        }
14        // 选择
15        path.push_back(items[i]);
16        // 递归
```

```

17         dfs_combination(items, path, i + 1, target - items[i], res);
18         // 回溯
19         path.pop_back();
20     }
21     // 组合问题为有约束条件的子集问题
22 }
23
24 vector<vector<int>> combination(const vector<int>& items, int target){
25     vector<int> path;
26     vector<vector<int>> res;
27     dfs_combination(items, path, 0, target, res);
28     return res;
29 }

```

- 全排列问题：有顺序，顺序不同算不同排列，必须用完所有元素，不能重复选。

代码块

```

1 // 全排列问题模板
2 void dfs_permute(const vector<int>& items, vector<int>& path, vector<bool>&
visited, vector<vector<int>>& res){
3     if(path.size() == items.size()){
4         res.push_back(path);
5         return;
6     }
7     for(int i = 0; i < items.size(); i++){
8         if(visited[i]){
9             continue;
10        }
11        // 选择
12        visited[i] = true;
13        path.push_back(items[i]);
14        // 递归
15        dfs_permute(items, path, visited, res);
16        // 回溯
17        path.pop_back();
18        visited[i] = false;
19    }
20 }
21
22 vector<vector<int>> permute(const vector<int>& items){
23     vector<int> path;
24     vector<vector<int>> res;
25     vector<bool> visited(items.size(), false);
26     dfs_permute(items, path, visited, res);
27     return res;

```

- 字符串全排列：

代码块

```

1  // 字符串全排列
2  void dfs_permute(const string& str, string& path, vector<bool>& visited,
    vector<string>& res){
3      if(path.size() == str.size()){
4          res.push_back(path);
5          return;
6      }
7      for(int i = 0; i < str.size(); i++){
8          if(visited[i]){
9              continue;
10         }
11         visited[i] = true;
12         path.insert(path.size(), 1, str[i]);
13         dfs_permute(str, path, visited, res);
14         path.erase(path.size() - 1, 1);
15         visited[i] = false;
16     }
17 }
18
19 vector<string> permute(const string& str){
20     vector<string> res;
21     string path;
22     vector<bool> visited(str.size(), false);
23     dfs_permute(str, path, visited, res);
24     sort(res.begin(), res.end());
25     return res;
26 }

```

- N皇后问题：

代码块

```

1  // N皇后问题模板
2  void dfs_NQueen(int N, int row, vector<int>& path, vector<bool>& col_used,
    vector<bool>& diag1, vector<bool>& diag2, vector<vector<int>>& res){
3      // row为行数, col_used为列标记数组, diag1为主对角线标记数组, diag2为副对角线标
    记数组
4      if(row == N){
5          res.push_back(path);
6          return;

```

```

7     }
8     for(int col = 0; col < N; col++){
9         int d1 = row - col + N - 1;
10        int d2 = row + col;
11        if(col_used[col] || diag1[d1] || diag2[d2]){
12            continue;
13        }
14        // 选择
15        col_used[col] = true;
16        diag1[d1] = true;
17        diag2[d2] = true;
18        path.push_back(col);
19        // 递归
20        dfs_NQueen(N, row + 1, path, col_used, diag1, diag2, res);
21        // 回溯
22        path.pop_back();
23        col_used[col] = false;
24        diag1[d1] = false;
25        diag2[d2] = false;
26    }
27 }
28
29 vector<vector<int>> NQueen(int N){
30     vector<int> path;
31     vector<vector<int>> res;
32     // 行数列数为N, 主对角线数副对角线数为2 * N - 1
33     vector<bool> col_used(N, false), diag1(2 * N - 1, false), diag2(2 * N -
34     1, false);
35     dfs_NQueen(N, 0, path, col_used, diag1, diag2, res);
36     return res;
37 }

```

9. 高级数据结构

9.1 二叉树

- 二叉树的定义：

代码块

```

1     typedef struct TreeNode {
2         char val;
3         struct TreeNode *left;
4         struct TreeNode *right;
5         TreeNode() : val(0), left(nullptr), right(nullptr) {}
6         TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}

```

```
7 }TreeNode;
```

- 二叉树的遍历：

- 访问节点：

代码块

```
1 // 访问节点
2 void visit(TreeNode* node){
3     cout << node->val << " ";
4 }
```

- 中序遍历：

代码块

```
1 // 中序遍历
2 void inorder(TreeNode* root){
3     if(root == nullptr) {
4         return;
5     }
6     inorder(root->left);
7     visit(root);
8     inorder(root->right);
9 }
```

- 前序遍历：

代码块

```
1 // 前序遍历
2 void preorder(TreeNode* root){
3     if(root == nullptr) {
4         return;
5     }
6     visit(root);
7     preorder(root->left);
8     preorder(root->right);
9 }
```

- 后序遍历：

代码块

```

1  // 后序遍历
2  void postorder(TreeNode* root){
3      if(root == nullptr) {
4          return;
5      }
6      postorder(root->left);
7      postorder(root->right);
8      visit(root);
9  }

```

- 层序遍历：

代码块

```

1  // 层序遍历
2  void levelorder(TreeNode* root){
3      if(root == nullptr) {
4          return;
5      }
6      queue<TreeNode*> q;
7      q.push(root);
8      while(!q.empty()){
9          TreeNode* node = q.front();
10         q.pop();
11         visit(node);
12         if(node->left != nullptr) {
13             q.push(node->left);
14         }
15         if(node->right != nullptr) {
16             q.push(node->right);
17         }
18     }
19 }

```

- 二叉树的构造：

- 利用中序遍历与前序遍历序列构造二叉树：

代码块

```

1  // 利用中序遍历序列与前序遍历序列构造树
2  TreeNode* buildTree_pre(string inorder, string preorder){
3      if(preorder.size() == 0){
4          return nullptr;
5      }
6      char ch = preorder[0];

```

```

7      // 确定根节点
8      TreeNode* root = new TreeNode(ch);
9      int pos = inorder.find(ch);
10     if(pos == string::npos){
11         return nullptr;
12     }
13     // 确定左子树和右子树的大小
14     string leftInorder = inorder.substr(0, pos), rightInorder =
inorder.substr(pos + 1);
15     int left_size = leftInorder.size(), right_size = rightInorder.size();
16     string leftPreorder = preorder.substr(1, left_size), rightPreorder =
preorder.substr(pos + 1, right_size);
17     // 构造左子树
18     root->left = buildTree_pre(leftInorder, leftPreorder);
19     // 构造右子树
20     root->right = buildTree_pre(rightInorder, rightPreorder);
21     return root;
22 }

```

- 利用中序遍历与后序遍历序列构造二叉树：

代码块

```

1  // 利用中序遍历序列与后序遍历序列构造树
2  TreeNode* buildTree_post(string inorder, string postorder){
3      if(postorder.size() == 0){
4          return nullptr;
5      }
6      char ch = postorder[postorder.size() - 1];
7      // 确定根节点
8      TreeNode* root = new TreeNode(ch);
9      int pos = inorder.find(ch);
10     if(pos == string::npos){
11         return nullptr;
12     }
13     // 确定左子树和右子树的大小
14     string leftInorder = inorder.substr(0, pos), rightInorder =
inorder.substr(pos + 1);
15     int left_size = leftInorder.size(), right_size = rightInorder.size();
16     string leftPostorder = postorder.substr(0, left_size),
rightPostorder = postorder.substr(pos, right_size);
17     // 构造左子树
18     root->left = buildTree_post(leftInorder, leftPostorder);
19     // 构造右子树
20     root->right = buildTree_post(rightInorder, rightPostorder);
21     return root;

```

9.2 二叉排序树

- 二叉排序树的定义：

代码块

```

1  // 二叉排序树
2  struct BST{
3      int val;
4      struct BST* left;    // 左子树
5      struct BST* right;   // 右子树
6      struct BST* parent; // 父节点
7      BST() : val(0), left(nullptr), right(nullptr), parent(nullptr) {}
8      BST(int _val, BST* _parent) : val(_val), left(nullptr), right(nullptr),
        parent(_parent){}
9  };

```

- 二叉排序树的遍历：

代码块

```

1  // 访问节点
2  void visit(BST* node){
3      cout << node->val << " ";
4  }
5
6  // 中序遍历
7  void inorder(BST* root){
8      if(root == nullptr){
9          return;
10     }
11     inorder(root->left);
12     visit(root);
13     inorder(root->right);
14 }
15
16 // 前序遍历
17 void preorder(BST* root){
18     if(root == nullptr){
19         return;
20     }
21     visit(root);
22     preorder(root->left);
23     preorder(root->right);

```

```

24 }
25
26 // 后序遍历
27 void postorder(BST* root){
28     if(root == nullptr){
29         return;
30     }
31     postorder(root->left);
32     postorder(root->right);
33     visit(root);
34 }
35
36 // 层序遍历
37 void levelorder(BST* root){
38     queue<BST*> q;
39     q.push(root);
40     while(!q.empty()){
41         BST* node = q.front();
42         q.pop();
43         visit(node);
44         if(node->left != nullptr){
45             q.push(node->left);
46         }
47         if(node->right != nullptr){
48             q.push(node->right);
49         }
50     }
51 }

```

- 二叉排序树的插入：

代码块

```

1 // 插入
2 BST* BST_insert(BST*& root, BST* _parent, int _val){ // 返回的是新建节点或目标节点
3     if(root == nullptr){
4         root = new BST(_val, _parent);
5         return root;
6     }
7     if(_val == root->val){
8         return root;
9     }else if(_val < root->val){
10         return BST_insert(root->left, root, _val);
11     }else{
12         return BST_insert(root->right, root, _val);

```

```
13     }
14 }
```

- 判断二叉排序树是否相等：

代码块

```
1  // 判断两棵树是否相同
2  bool isSameBST(BST* root1, BST* root2) {
3      if (root1 == nullptr && root2 == nullptr) {
4          return true;
5      } else if ((root1 == nullptr && root2 != nullptr) || (root1 != nullptr
&& root2 == nullptr)) {
6          return false;
7      } else {
8          if (root1->val != root2->val) {
9              return false;
10             } else {
11                 return isSameBST(root1->left, root2->left) && isSameBST(root1-
>right, root2->right);
12             }
13     }
14 }
```

9.3 优先队列priority_queue

- priority_queue的使用（包含在<queue>库中）：

代码块

```
1  int main(){
2      priority_queue<int> pq;
3      // 判空与长度
4      cout << pq.empty() << " " << pq.size() << endl;
5      // 添加
6      for(int i = 0; i < 10; i++){
7          pq.push(i);
8      }
9      // 删除
10     pq.pop(); // 删除优先级最高的元素
11     // 访问
12     cout << pq.top() << endl; // 取优先级最高的元素
13 }
```

- priority_queue优先级自定义：

- 使用重载运算符：

代码块

```
1 // 定义结构体：存储学生的姓名和分数
2 struct Student {
3     string name;
4     int score;
5
6     // 重载 < 运算符：分数越高，优先级越高（大顶堆）
7     bool operator<(const Student& other) const {
8         // 规则：当前对象的score < other的score → other优先级更高
9         return score < other.score;
10    }
11 };
12
13 int main() {
14     priority_queue<Student> q;
15
16     q.push({"Alice", 85});
17     q.push({"Bob", 92});
18     q.push({"Charlie", 78});
19
20     // 输出: Bob(92) → Alice(85) → Charlie(78)
21     while (!q.empty()) {
22         auto s = q.top();
23         cout << s.name << "(" << s.score << ")" ";
24         q.pop();
25     }
26     return 0;
27 }
```

- 使用结构体：

代码块

```
1 struct Compare_Int{
2     bool operator()(const int x, const int y){
3         return x > y;    // 小根堆，值越小优先级越高
4     }
5 };
6
7 int main(){
8     priority_queue<int, vector<int>, Compare_Int> pq;
9     int n, val;
10    cin >> n;
11    for(int i = 0; i < n; i++){
```

```

12         cin >> val;
13         pq.push(val);
14     }
15 }

```

- 哈夫曼树:

代码块

```

1  // 哈夫曼树定义
2  struct HTNode {
3      int weight;
4      struct HTNode* left;
5      struct HTNode* right;
6      HTNode() : weight(0), left(nullptr), right(nullptr) {}
7      HTNode(int w, HTNode* l, HTNode* r) {
8          weight = w;
9          left = l;
10         right = r;
11     }
12     HTNode(HTNode* l, HTNode* r) {
13         weight = l->weight + r->weight;
14         left = l;
15         right = r;
16     }
17 };
18
19 // 优先级队列比较函数
20 struct Compare_HT_ptr {
21     bool operator()(const HTNode* h1, const HTNode* h2) {
22         return h1->weight > h2->weight;
23     }
24 };
25
26 // 创建哈夫曼树
27 HTNode* createHTree(const vector<int>& weight) {
28     priority_queue<HTNode*, vector<HTNode*>, Compare_HT_ptr> pq;
29     for (int w : weight) {
30         auto* h = new HTNode(w, nullptr, nullptr);
31         pq.push(h);
32     }
33     while (pq.size() != 1) {
34         HTNode* h1 = pq.top();
35         pq.pop();
36         HTNode* h2 = pq.top();
37         pq.pop();

```

```

38         auto* new_h = new HTNode(h1, h2);
39         pq.push(new_h);
40     }
41     HTNode* root = pq.top();
42     pq.pop();
43     return root;
44 }
45
46 // 计算哈夫曼树编码的权值和
47 int calculateWPL(HTNode* root, int h) {
48     if (root == nullptr) {
49         return 0;
50     } else if (root->left == nullptr && root->right == nullptr) {
51         return h * root->weight;
52     } else {
53         return calculateWPL(root->left, h + 1) + calculateWPL(root->right,
54             h + 1);
55     }
56 }

```

9.4 哈希表

9.4.1 unordered_set与set



如果需要有序则使用set（按键升序，红黑树实现），无序则使用unordered_set（哈希表实现）。

- unordered_set的定义：主要用于判重（如BFS）

代码块

```

1  int main(){
2      unordered_set<int> set_int; // 存int型
3      unordered_set<string> set_str; // 存字符串
4  }

```

- unordered_set的使用：

代码块

```

1  int main(){
2      unordered_set<string> set;
3      // 插入
4      set.insert("hello");

```

```

5     set.insert("world");
6     // 查找
7     if(set.find("hello") != set.end()){ // 查找失败时返回set.end()
8         cout << "hello" << endl;
9     }
10    if(set.count("world")){ // 查找失败时返回0
11        cout << "world" << endl;
12    }
13    // 删除
14    set.erase("hello");
15    // 清空
16    set.clear();
17 }

```

9.4.2 unordered_map与map



如果需要有序则使用map（按键升序，红黑树实现），无序则使用unordered_map（哈希表实现）。

- unordered_map的定义：

代码块

```

1  int main(){
2      // 示例1: int键 + int值 (最常用, 比如统计数字频次)
3      unordered_map<int, int> mp1;
4      // 示例2: string键 + int值 (比如统计字符串频次)
5      unordered_map<string, int> mp2;
6      // 示例3: char键 + string值
7      unordered_map<char, string> mp3;
8  }

```

- unordered_map的使用：

代码块

```

1  int main(){
2      unordered_map<string, int> map; // 键常用int/string/char
3      // 直接赋值
4      map["hello"] = 1; // 没有该键则插入
5      // 插入
6      map.insert(pair<string, int>("world", 2));
7      // 访问
8      cout << map["hello"] << endl; // 按键访问

```

```

9      // 遍历
10     for(auto & pair : map){ // 键值对遍历
11         cout << pair.first << ":" << pair.second << endl;
12     }
13     // 删除
14     map.erase("hello"); // 按键删除
15     // 查找
16     unordered_map<string, int>::iterator it = map.find("world");
17     if(it != map.end()){ // 查找失败时返回map.end()
18         cout << it->first << ":" << it->second << endl;
19     }
20     // 清空
21     map.clear();
22 }

```

- map的使用：

代码块

```

1  int main(){
2      map<string, int> map;
3      // 插入
4      map.insert(pair<string, int>("hello", 1));
5      map.insert(pair<string, int>("world", 2));
6      // 访问
7      for(auto & pair : map){ // 键值对遍历
8          cout << pair.first << ":" << pair.second << endl;
9      }
10     // 查找
11     auto it = map.find("world");
12     if(it != map.end()){ // 删除失败时返回map.end()
13         cout << it->first << ":" << it->second << endl;
14     }
15     // 获取迭代器
16     auto first = map.begin(); // 返回键值对的第一个元素
17     auto last = map.rbegin(); // 返回键值对最后一个元素
18     // 删除
19     map.erase("hello");
20     // 清空
21     map.clear();
22 }

```

10. 动态规划

10.1 递推求解

- 爬楼梯问题：

- 初始版本：一次可以走1个台阶也可以走2个台阶，走n级台阶的可能种数： $dp[n]=dp[n-1]+dp[n-2]$ 。

代码块

```
1 // 爬楼梯问题
2 int climb_stairs(int n) {
3     if (n <= 1) {
4         return 1;
5     }
6     vector<int> dp(n + 1);
7     dp[0] = 1;
8     dp[1] = 1;
9     for (int i = 2; i <= n; ++i) {
10         dp[i] = dp[i-1] + dp[i-2];
11     }
12     return dp[n];
13 }
```

- 进阶版本：一次可以走1个、2个或3个台阶，走n级台阶的可能种数： $dp[n]=dp[n-1]+dp[n-2]+dp[n-3]$ 。

代码块

```
1 // 爬楼梯问题进阶版
2 int climb_stairs_plus(int n) {
3     vector<int> dp(n + 1);
4     dp[0] = 1;
5     dp[1] = 1;
6     dp[2] = 2;
7     for (int i = 3; i <= n; ++i) {
8         dp[i] = dp[i-1] + dp[i-2] + dp[i-3];
9     }
10     return dp[n];
11 }
```

- 子数组和问题：

- 最大子数组和： $dp[i] = \max(nums[i], dp[i-1] + nums[i])$ 。

代码块

```
1 // 最大子数组和
2 int maxSubArray(const vector<int>& nums) {
3     int n = nums.size();
```

```

4     vector<int> dp(n);
5     dp[0] = nums[0];
6     int res = dp[0];
7     for(int i = 1; i < n; i++){
8         dp[i] = max(nums[i], dp[i-1] + nums[i]);
9         res = max(res, dp[i]);
10    }
11    return res;
12 }

```

- 最小子数组和： $dp[i] = \min(nums[i], dp[i-1] + nums[i])$ 。

代码块

```

1  // 最小子数组和
2  int minSubArray(const vector<int>& nums){
3      int n = nums.size();
4      vector<int> dp(n);
5      dp[0] = nums[0];
6      int res = dp[0];
7      for(int i = 1; i < n; i++){
8          dp[i] = min(nums[i], dp[i-1] + nums[i]);
9          res = min(res, dp[i]);
10     }
11     return res;
12 }

```

10.2 背包问题

- 0-1背包问题：每个物品只能拿一次（逆序）。
 - 取最大值：

代码块

```

1  // 0-1背包取最大值
2  int zero_one_bag_max(vector<int>& weight, vector<int>& value, int C) {
3      vector<int> dp(C + 1, 0);
4      dp[0] = 0;
5      for (int i = 0; i < weight.size(); i++) {
6          for (int j = C; j >= weight[i]; j--) {
7              dp[j] = max(dp[j], dp[j - weight[i]] + value[i]);
8          }
9      }
10     return dp[C];

```

```
11 }
```

- 取最小值：

代码块

```
1 // 0-1背包取最小值
2 int zero_one_bag_min(vector<int>& weight, vector<int>& value, int C){
3     vector<int> dp(C + 1, INT_MAX);
4     dp[0] = 0;
5     for(int i = 0; i < weight.size(); i++){
6         for(int j = C; j >= weight[i]; j--){
7             if(dp[j - weight[i]] != INT_MAX){ // 防止越界
8                 dp[j] = min(dp[j], dp[j - weight[i]] + value[i]);
9             }
10        }
11    }
12    return dp[C];
13 }
```

- 完全背包问题：每个物品可以拿多次（正序）。

- 取最大值：

代码块

```
1 // 完全背包取最大值
2 int complete_bag_max(vector<int>& weight, vector<int>& value, int C) {
3     vector<int> dp(C + 1, 0);
4     dp[0] = 0;
5     for (int i = 0; i < weight.size(); i++) {
6         for (int j = weight[i]; j <= C; j++) {
7             dp[j] = max(dp[j], dp[j - weight[i]] + value[i]);
8         }
9     }
10    return dp[C];
11 }
```

- 取最小值：

代码块

```
1 // 完全背包取最小值
2 int complete_bag_min(vector<int>& weight, vector<int>& value, int C){
3     vector<int> dp(C + 1, INT_MAX);
```

```

4     dp[0] = 0;
5     for(int i = 0; i < weight.size(); i++){
6         for(int j = weight[i]; j <= C; j++){
7             if(dp[j - weight[i]] != INT_MAX){ // 防止越界
8                 dp[j] = min(dp[j], dp[j - weight[i]] + value[i]);
9             }
10        }
11    }
12    return dp[C];
13 }

```

11. 图论

11.1 图的定义与遍历

- 图的定义：

代码块

```

1 // 图的邻接矩阵表示
2 struct Graph{
3     vector<vector<int>> matrix; // 邻接矩阵
4     vector<int> nodes; // 顶点向量
5     int vex_num; // 顶点数
6     int arc_num; // 边数
7     bool graph_type; // true为有向图, false为无向图
8
9     Graph(int num, bool type){
10         vex_num = num;
11         arc_num = 0;
12         graph_type = type;
13         matrix = vector<vector<int>>(num, vector<int>(num, 0));
14         nodes = vector<int>(num, 0);
15     }
16
17     Graph(bool type, vector<int> items){
18         vex_num = items.size();
19         arc_num = 0;
20         graph_type = type;
21         nodes = items;
22         matrix = vector<vector<int>>(items.size(), vector<int>
23 (items.size(), 0));
24
25     void addEdge(int x, int y){ // 添加边
26         if (x < 0 || y < 0 || x >= vex_num || y >= vex_num){

```

```

27         return;
28     }
29     if(graph_type){
30         matrix[x][y] = 1;
31     }else{
32         matrix[x][y] = 1;
33         matrix[y][x] = 1;
34     }
35     arc_num++;
36 }
37
38 void addNode(int node){    // 添加顶点
39     nodes.push_back(node);
40     for(int i = 0; i < vex_num; i++){
41         matrix[i].push_back(0);
42     }
43     matrix.push_back(vector<int>(vex_num + 1, 0));
44     vex_num++;
45 }
46
47 void print(){            // 打印邻接矩阵
48     for(int i = 0; i < vex_num; i++){
49         for(int j = 0; j < vex_num; j++){
50             cout << matrix[i][j] << " ";
51         }
52         cout << endl;
53     }
54 }
55 };

```

- 图的遍历：

代码块

```

1  // 访问顶点
2  void visit(const Graph& graph, int index){
3      cout << graph.nodes[index] << " ";
4  }
5
6  // 深度优先搜索
7  void dfs(const Graph& graph, vector<bool>& visited, int start){
8      visit(graph, start);
9      visited[start] = true;
10     for(int end = 0; end < graph.vex_num; end++){
11         if(graph.matrix[start][end] && !visited[end]){
12             dfs(graph, visited, end);

```

```

13     }
14 }
15 }
16
17 void DFS(const Graph& graph){
18     vector<bool> visited(graph.vex_num, false);
19     for(int i = 0; i < graph.vex_num; i++){
20         if(!visited[i]){
21             dfs(graph, visited, i);
22         }
23     }
24 }
25
26 // 广度优先搜索
27 void bfs(const Graph& graph, vector<bool>& visited, int start){
28     queue<int> q;
29     q.push(start);
30     visited[start] = true;
31     while(!q.empty()){
32         int index = q.front();
33         q.pop();
34         visit(graph, index);
35         for(int end = 0; end < graph.vex_num; end++){
36             if(graph.matrix[index][end] && !visited[end]){
37                 q.push(end);
38                 visited[end] = true;
39             }
40         }
41     }
42 }
43
44 void BFS(const Graph& graph){
45     vector<bool> visited(graph.vex_num, false);
46     for(int i = 0; i < graph.vex_num; i++){
47         if(!visited[i]){
48             bfs(graph, visited, i);
49         }
50     }
51 }

```

11.2 并查集

- 并查集的定义与基本操作：

代码块

```
1  const int MAX_SIZE = 100;
```

```
2
3 // 并查集定义
4 struct UFSet{
5     int parent[MAX_SIZE]; // 父节点
6     int size;             // 集合大小
7 };
8
9 // 初始化并查集
10 void Init(UFSet& set, int n) {
11     for (int i = 0; i < n; i++) {
12         set.parent[i] = i; // 初始化父节点为自身
13     }
14     set.size = n;
15 }
16
17 // 查找根节点
18 int Find(UFSet& set, int x) {
19     if (set.parent[x] != x) { // 递归寻找根节点
20         set.parent[x] = Find(set, set.parent[x]);
21     }
22     return set.parent[x];
23 }
24
25 // 合并两个集合
26 void Union(UFSet& set, int x, int y) {
27     int x_root = Find(set, x);
28     int y_root = Find(set, y);
29     if (x_root != y_root) { // 不同集合
30         set.parent[x_root] = y_root;
31         set.size--;
32     }
33 }
```